IMT Atlantique

Mixed Reality

BENTATA Hugo - SCHOBERT
Néo - Montadhar ETTAIEB

Projet RVRA

Table des matières

I Concept Description **2**

II Hardware Configuration **2**

III Interaction Description **2**

IV Proposed Software Architecture **3**

IV.1 Core Managers (Global State) 3

IV.2 Player Controllers (Input & Tracking) 3

IV.3 Interactive & Networked Elements 3

IV.4 Data Flow 4

V Git Repository **4**

Application v1 Summary Document :
”Escape the Island”

I Concept Description

”Escape the Island” is an asymmetric cross-reality (VR/AR) multiplayer death run game. The game takes place in a shared synchronized environment (Little St. James island and temple) and is divided into two distinct acts :

- **Act 1 : The Infiltration :** The VR user spawns on the edge of the island and must navigate to the temple door. The AR user views the island from a bird’s-eye perspective using an Android smartphone. The AR user tries to locate the VR user while he makes his escape. He can attack the AR user by tapping the screen to drop bombs.
- **Act 2 : The Boss Fight :** Once the VR user reaches the temple door and presses the button, the session transitions. The AR user is revealed to be the boss (Epstein) and is teleported into the same room. The VR user receives a weapon and must shoot the AR user within a time limit, while the AR user must physically dodge the incoming fire using their phone’s positional tracking.

II Hardware Configuration

- **VR Platform :** Meta Quest 2
- **AR Platform :** Android smartphone with ARCore

III Interaction Description

Individual Interactions

VR User : First-person locomotion (joystick/teleport) to navigate the map. Controller input to press buttons and aim/fire a weapon.

AR User : 6DOF spatial tracking to change the camera angle and view the miniature island map. Touchscreen input (tapping the screen) to spawn bombs. Physical evasion (swaying the phone in the real world) to dodge attacks in Phase 2.

Shared / Cross-Platform Interactions

- **Asymmetric Adversarial Interaction :** Real-time spatial synchronization where the AR user's touch inputs dynamically spawn physics objects (bombs) directly into the VR user's path.
- **Synchronized Hitboxes :** In the second phase, the VR user's projectile trajectories are synced over the network to interact directly with the AR user's physical device position.

IV Proposed Software Architecture

(This is a very early first draft, with heavy inspiration from the previous lab session, subject to change)

The application will use a modular, component-based architecture relying on a **Host-Client** networking model (using a framework like Unity Netcode). The VR headset will likely act as the Host (authoritative over physics and state), while the AR device connects as a Client.

The codebase will be structured around the following core modules :

IV.1 Core Managers (Global State)

- **GameManager :** A networked singleton responsible for tracking the global game state. It listens for the VR user pressing the temple button and triggers the transition from Phase 1 to Phase 2.
- **NetworkManager :** Manages the connection lobby, synchronizes player transforms, and handles the networked instantiation of shared objects.

IV.2 Player Controllers (Input & Tracking)

- **VR_PlayerController :** Handles local VR inputs (OVR/XR Rig). Processes locomotion and manages hand-tracking for aiming in Phase 2.
- **AR_PlayerController :** Translates ARCore spatial tracking data into a virtual position. Includes a `TouchInputHandler` that casts rays to calculate coordinates for spawning hazards.

IV.3 Interactive & Networked Elements

- **BombSpawner & BombController :** Sends an RPC (Remote Procedure Call) to the Host to instantiate a bomb prefab. Manages gravity, timers, and collision.
- **WeaponController & Projectile :** Spawns networked projectiles that calculate collisions against the AR player's networked hitbox.

IV.4 Data Flow

Input is processed locally on each device. The AR device sends command inputs to the VR Host. The Host calculates the physics/collisions and synchronizes the updated positions and states back to the AR Client.

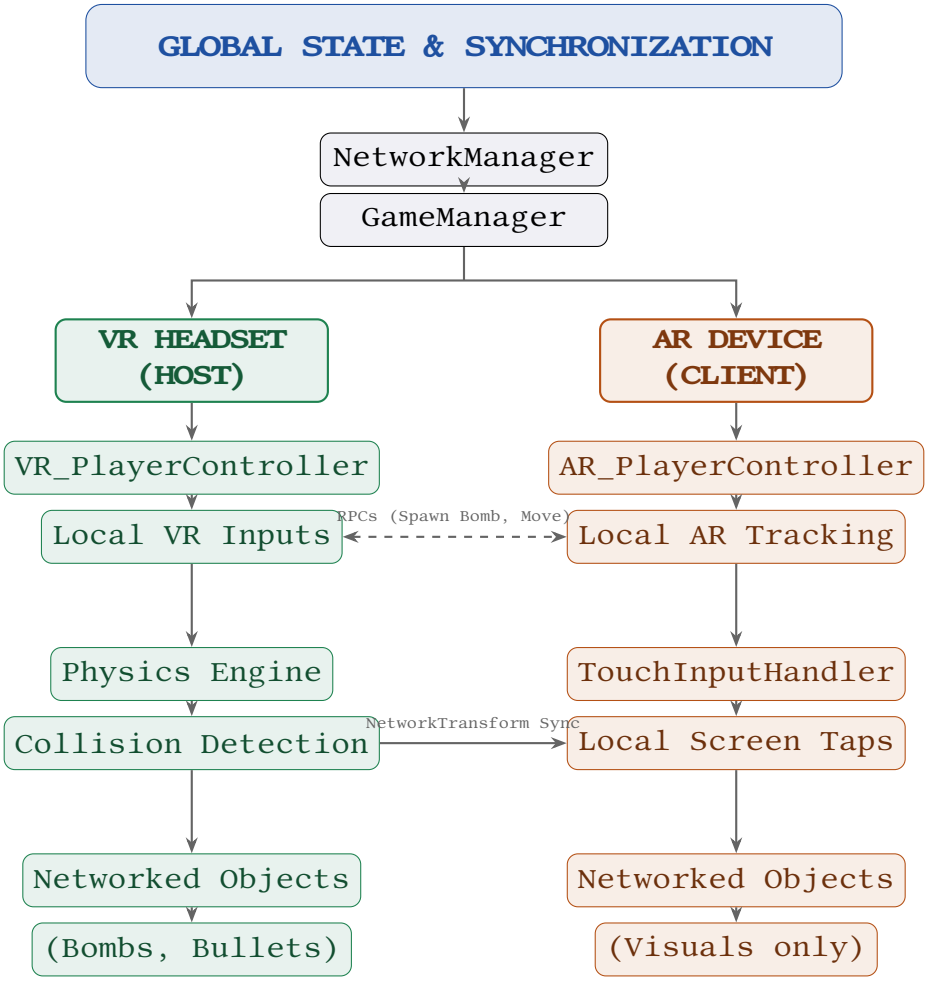

Figure 1 – Software Architecture — Data Flow Diagram

V Git Repository

- **Repository Link :** github.com/neo-schobert/Projet-Final-RVRA
- **Current Tag :** `v0-init`